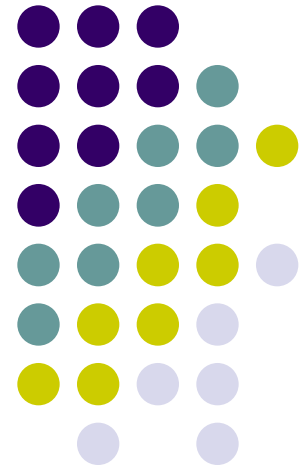


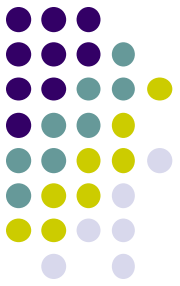
Seguridad Industrial

Unidad 6

Ingeniería de Seguridad de Software. Errores Humanos y Riesgos

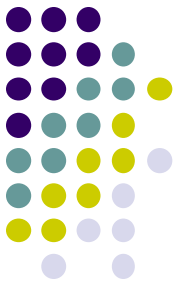


Introducción



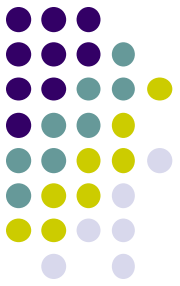
- El problema del software inseguro es uno de los retos técnico más importantes de nuestros tiempos.
- La seguridad es uno de los factores claves limitantes sobre los que podemos crear con la tecnología de la información.
- El aumento de la concientización de seguridad ha hecho que muchas organizaciones implementen controles de seguridad de red y sistemas operativos en forma cada vez más eficientes.
- Como contrapartida los atacantes han migrado los ataques hacia las aplicaciones, en su mayoría web y móviles.

Seguridad en el Ciclo de Desarrollo de Software



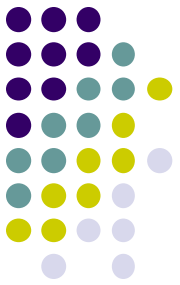
- Para prevenir problemas de seguridad en una aplicación, es esencial implementar la seguridad dentro del Ciclo de Vida de Desarrollo del Software (en inglés Systems Development Life Cycle o SDLC), desarrollando estándares, políticas y guías de uso que encajen y funcionen dentro de la metodología de desarrollo.
- Mediante la integración de la seguridad en cada fase del SDLC, permite una aproximación integral a la seguridad de aplicaciones que se apoya en los procedimientos ya existentes en la organización.

Modelo testear y parchar

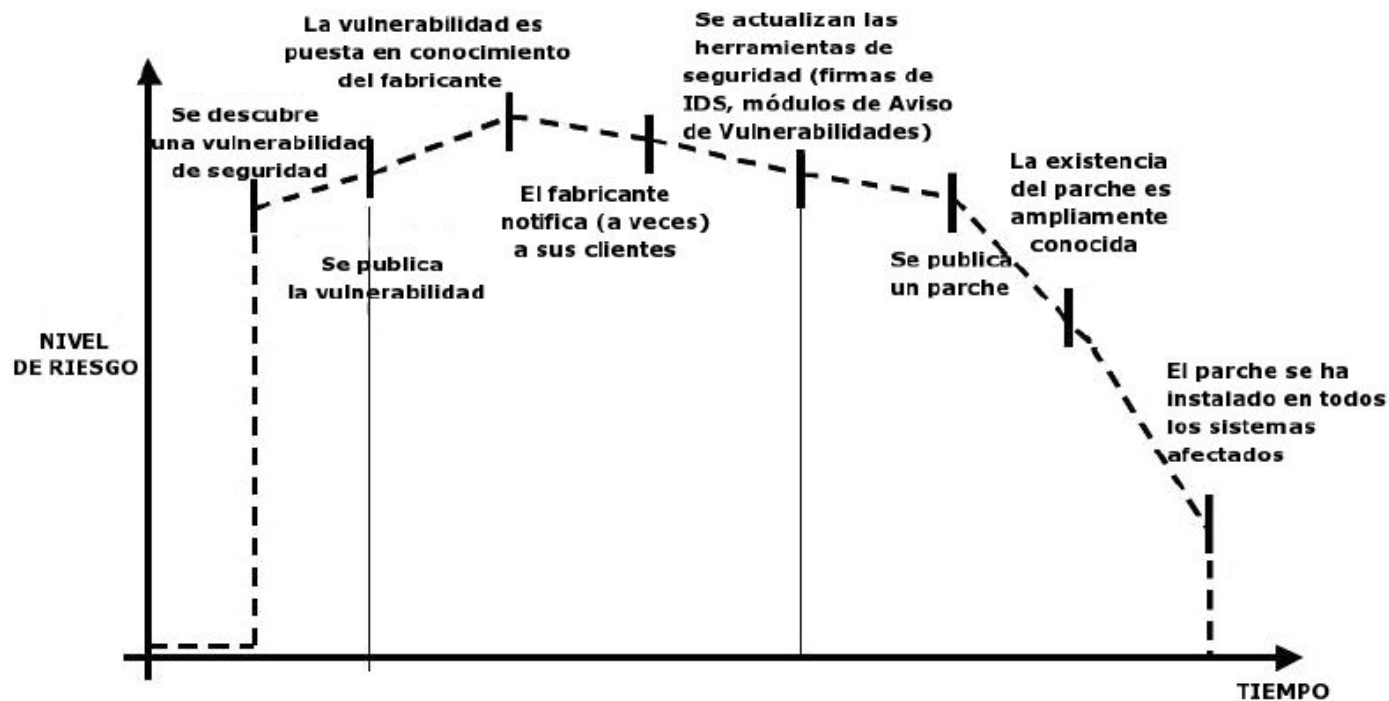


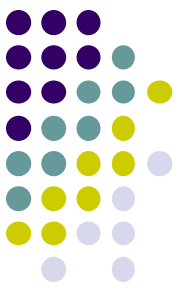
- El modelo de testear y parchear comprende corregir un bug reportado, pero sin la investigación adecuada de la causa origen.
- La evolución de vulnerabilidades en el software común usado por todo el mundo ha demostrado la ineficacia de este modelo, ya que con el tiempo de reacción de los atacantes por todo el mundo, la ventana de vulnerabilidad típica no provee del tiempo suficiente para la instalación de parches, ya que el tiempo entre que la vulnerabilidad es descubierta y un ataque automatizado es desarrollado y divulgado decrece cada año.

Modelo testear y parchar



- El modelo de testear y parchear se asocia a menudo con la ventana de vulnerabilidad.

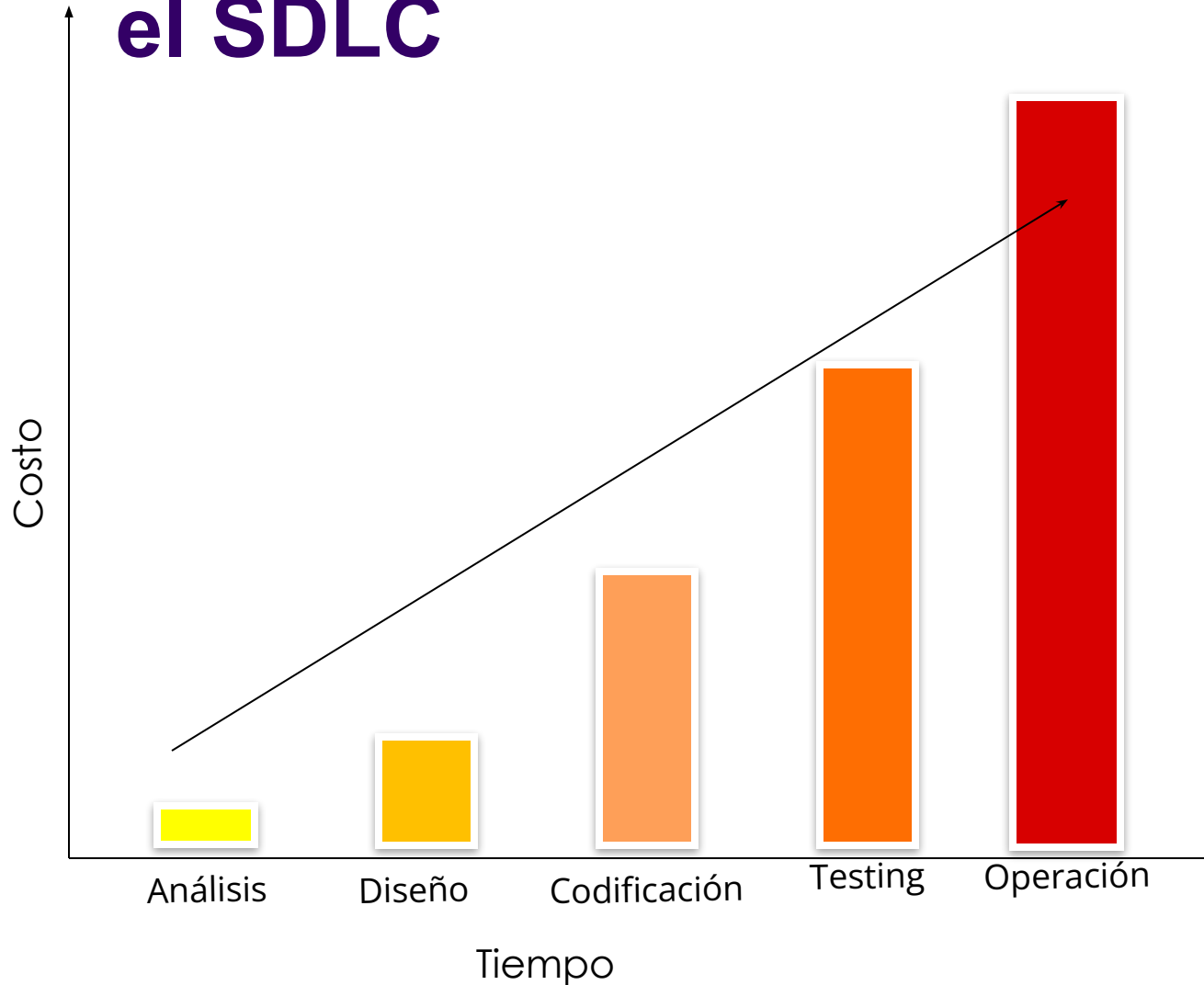
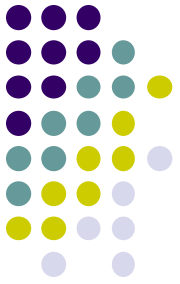




Cuanto antes mejor

- Detectar un bug en una etapa temprana dentro del SDLC permite que pueda ser abordado con mayor rapidez y a un coste menor.
- Es importante comprender que un bug de seguridad no es diferente de uno funcional o de rendimiento.
- A pesar de que las nuevas bibliotecas, herramientas o lenguajes pueden ayudar a diseñar mejores programas (con menos bugs de seguridad), nuevas amenazas están apareciendo constantemente y los desarrolladores deben ser conscientes de aquellas que afectan al software que están desarrollando.
- Un factor clave es capacitar a los equipos de desarrollo y calidad sobre problemas de seguridad comunes y los métodos para detectar y prevenirlos.

Costo de la Seguridad en el SDLC

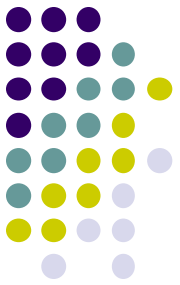


Pruebas de Seguridad en aplicaciones



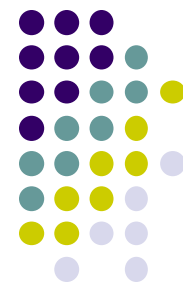
- En la actualidad la mayoría de personas realizan pruebas a la tecnología o el propio software, pero no realizan pruebas al software hasta que ya ha sido creado está en la fase de desarrollo de su ciclo de vida (es decir, que el código ha sido creado e instanciado en una aplicación web en uso).
- Generalmente, esta es una práctica muy ineficaz y de alto costo, por ello es que revisaremos algunos principios básicos que deberían ser tenidos en cuenta por los profesionales a la hora de realizar pruebas en busca de bugs de seguridad en software.
- Siempre debemos tener en cuenta que:
“la seguridad es un proceso, no un producto”

Comprende el objeto de estudio



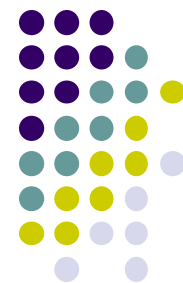
- Una de las primeras iniciativas principales en cualquier buen programa de seguridad debería ser pedir documentación precisa de la aplicación.
- La arquitectura, diagramas de flujo de datos, casos de uso, y demás deberían ser escritos en documentos formales y puestos a disposición para revisión.
- La documentos de aplicación y especificación técnica deberían incluir información que liste no solo los casos de uso deseados, sino también cualquier caso de uso específicamente no admitido.

Utiliza las herramientas adecuadas



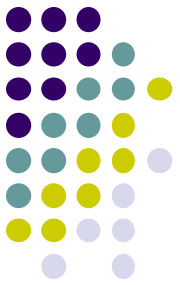
- Aunque no existe una herramienta “bala de plata”, las herramientas juegan un papel crítico en el programa de seguridad global.
- Hay toda una gama de herramientas de código abierto y comerciales que pueden ayudar en la automatización de muchas tareas de seguridad rutinarias.
- Estas herramientas pueden simplificar y acelerar el proceso de seguridad ayudando al personal de seguridad en sus tareas.
- Sin embargo, es importante comprender exactamente lo que estas herramientas pueden hacer y lo que no, de forma que no se exagere su uso o sean usadas incorrectamente.

Usa el código fuente cuando esté disponible

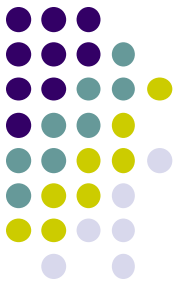


- A pesar de los resultados de los tests de penetración de caja negra pueden ser impresionantes y útiles para demostrar como son expuestas las vulnerabilidades en producción, no son el método más efectivo para securizar una aplicación.
- Si el código fuente de la aplicación está disponible, debería ser entregado al personal de seguridad para ayudarles durante la realización de la revisión, ya que es posible descubrir vulnerabilidades dentro del código fuente de la aplicación que se perderían durante el trabajo de caja negra.

Desarrolle métricas

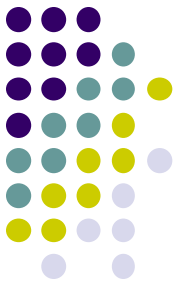


- Una parte importante de un buen programa de seguridad es la habilidad para determinar si las cosas están mejorando.
- Es importante seguir la pista de los resultados de los trabajos de prueba, y desarrollar métricas que podrán revelar la evolución de la seguridad de la aplicación en la organización.
- Estas métricas pueden mostrar si son necesarias más educación y formación, si hay algún mecanismo de seguridad en particular que no es comprendido con claridad por desarrollo, y si el número total de problemas relacionados con seguridad que se han encontrado cada mes se está reduciendo.



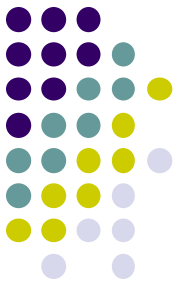
Técnicas de comprobación de seguridad

- A continuación se describe una visión genérica de alto nivel de las diversas técnicas de testing que pueden ser empleadas en la construcción de un programa de pruebas. Ellas son:
 - Inspecciones y Revisiones Manuales
 - Modelado de Amenazas
 - Revisión de Código
 - Pruebas de Penetración



Inspecciones y revisiones manuales

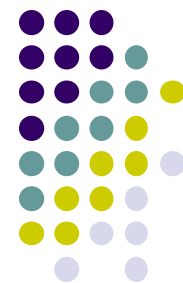
- Las inspecciones manuales son revisiones realizadas por personas, que comprueban las implicaciones de seguridad de personas, políticas y procesos, aunque pueden incluir la inspección de aspectos tecnológicos, como puede ser los diseños de la arquitectura escogidos.
- Son particularmente buenas para comprobar si las personas comprenden el proceso de seguridad, si han sido concienciadas de la existencia de una política, y si tienen los conocimientos apropiados para diseñar y/o implementar una aplicación segura.
- Esto incluye la revisión manual de documentación, políticas de programación segura, requerimientos de seguridad y diseños de arquitectura entre otros.



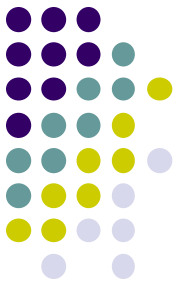
Modelado de amenazas

- El modelado de amenazas se ha convertido en una técnica popular para ayudar a los diseñadores de sistemas acerca de las amenazas de seguridad a las que se enfrentan sus sistemas, ya que permiten concentrar los recursos limitados en aquellas partes del sistema que más lo necesitan.
- El modelado de amenazas es esencialmente la **evaluación del riesgo en aplicaciones**, por lo tanto se recomienda que todas las aplicaciones tengan un modelo de amenaza desarrollado y documentado.

Modelado de amenazas



- Para desarrollar un modelo de amenaza, recomendamos tomar una aproximación que incluya los siguiente pasos (según estandar NIST 800-30):
 - Descomponer la aplicación a través de un proceso de inspección manual, comprendiendo como funciona la aplicación, sus activos, funcionalidad y conectividad.
 - Definir y clasificar los activos en activos tangibles e intangibles, y ponderarlos de acuerdo a su criticidad para el negocio.
 - Explorar vulnerabilidades potenciales (técnicas, operacionales y de gestión)
 - Explorar amenazas potenciales a través de un proceso de desarrollo de escenarios de amenaza o árboles de ataque que desarrollan una visión realista de potenciales vectores de ataque desde la perspectiva del atacante.
 - Crear estrategias de mitigación y desarrollar para cada una de las amenazas que se consideran realistas.



Revisión de código fuente

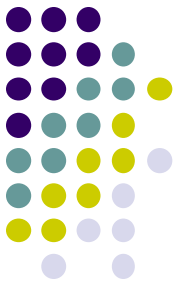
- Es el proceso de comprobar el código fuente de una aplicación web en busca de incidencias de seguridad, ya sea en forma manual o con herramientas automatizadas
- Muchos problemas de seguridad significativos, intencionados o no, son extremadamente difíciles de descubrir con otras formas, lo que hace del análisis de código la técnica preferida para las comprobaciones técnicas.
- Casi todos los expertos en seguridad están de acuerdo en que no hay nada mejor que ver el código ya que toda la información necesaria para identificar problemas de seguridad está en algún lugar del código.

Pruebas de intrusión

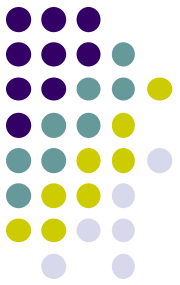


- Los tests de intrusión (PenTest) se han convertido desde hace muchos años en una técnica común empleada para comprobar la seguridad de una red.
- Las pruebas de intrusión se basan en la comprobación de una aplicación en ejecución remota sin saber el funcionamiento interno de la aplicación, para encontrar vulnerabilidades de seguridad.
- Es por esta razón que si bien este tipo de comprobación tienen un importante aporte en un programa de pruebas, no deben ser consideradas como la principal o única técnica a utilizar.

La necesidad de un enfoque equilibrado



- A la hora de tomar decisiones, la experiencia muestra que todas las técnicas deberían ser empleadas en lo posible para asegurar que todas las áreas que necesitan ser probadas son cubiertas.
- Muchas compañías adoptan a las pruebas de penetración como estrategia de test, pero este enfoque no puede abordar efectivamente muchas de las incidencias que requieren ser comprobadas, y simplemente es insuficiente y demasiado tardía en el ciclo de desarrollo del software.
- Una aproximación correcta debe asegurar la cobertura de pruebas en todas las fases del SDLC ,explotando el potencial de las técnicas más apropiadas disponibles dependiendo de la fase actual del SDLC.
- Este equilibrio varía dependiendo de muchos factores, como la madurez del proceso de pruebas y la cultura corporativa, aunque es esencial para las compañías poner un mayor énfasis en las etapas iniciales de desarrollo.

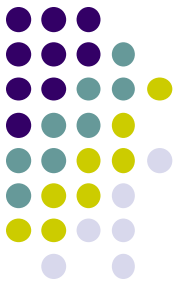


Seguridad Industrial

Unidad 6

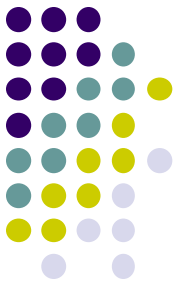
Ingeniería de Software para
Sistemas Seguros

Principios de desarrollo seguro



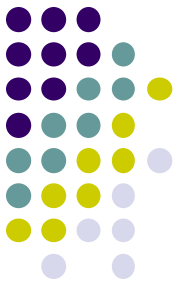
Principios de desarrollo seguro

- Minimizar superficies de ataque
- Seguridad por defecto
- Mínimo privilegio
- Defensa en profundidad
- Fallar de manera segura
- Los sistemas externos son inseguros
- No confiar en la seguridad por obscuridad
- Simplicidad
- Arreglar las cuestiones de seguridad correctamente



Minimizar la superficie de ataque

- Cada característica que se añade a una aplicación agrega una cierta cantidad de riesgo al total.
- El objetivo del desarrollo seguro es reducir el riesgo total minimizando las posibles áreas de ataque.
 - *Por ejemplo, si una aplicación web implementa ayuda en línea con una función de búsqueda, la misma podría ser vulnerable a ataques de inyección SQL.*
 - *Si la ayuda se hubiera limitado a usuarios autorizados, la probabilidad del ataque se reduce y si además se agregaran rutinas de validación de datos centralizadas, la posibilidad de sufrir ataques de inyección SQL se reducen aún más.*
 - *Pero si la función de ayuda fuera reescrita sin la opción de búsqueda (por una interfaz de usuario mejorada, por ejemplo), se eliminaría por completo la posibilidad de este ataque.*



Seguridad por defecto

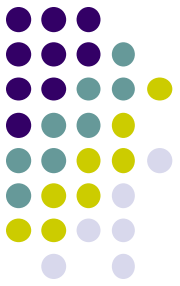
- Hay muchas maneras de entregar una aplicación “lista para usar” a los usuarios.
- En este caso la aplicación debería ser configurada en forma segura por defecto, y debería depender del usuario el reducir su nivel de seguridad, si lo requiere.
- *Por ejemplo, por defecto, debe habilitarse la complejidad de la contraseña y su duración, aunque luego a ciertos usuarios se les puede permitir deshabilitar esas dos características para simplificar su uso de la aplicación e incrementar su riesgo.*

Principio del mínimo privilegio

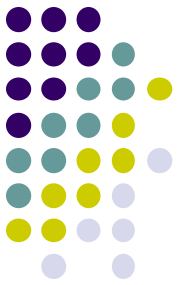


- El principio del mínimo privilegio recomienda que un objeto tenga siempre la mínima cantidad de privilegios necesarios para poder ejecutar la función de negocio para la cual fue creado.
- Esto abarca a los derechos de usuario, permisos de recursos tales como límites de CPU, memoria, red y permisos del sistema de archivos.
- *Por ejemplo, si un servidor middleware requiere acceso de lectura a la tabla de una base de datos, y la posibilidad de escribir en un log, se le debería otorgar solamente esos permisos y bajo ninguna circunstancia deberían darse por ejemplo privilegios de administrador.*

Principio de defensa en profundidad



- El principio de defensa en profundidad se basa en la teoría que cuanto más controles se poseen contra diferentes tipos de riesgo, mayor nivel de defensa se logra.
- Cuando se utilizan controles en profundidad, se hace extraordinariamente difícil la explotación de vulnerabilidades y por lo tanto existe poca probabilidad de que los ataques puedan ocurrir.
- Con la codificación segura, esto puede tomar por ejemplo, rutinas de validación, controles de auditoria centralizados, y requerir a los usuarios hacer login en todas las páginas.
- *Por ejemplo, una interfaz administrativa con defectos es poco probable que sea vulnerable a ataques anónimos si incorpora correctamente el control de acceso, chequea la autorización administrativa del usuario, y genera un log de todos los accesos.*



Fallar de manera segura

- Las aplicaciones pueden fallar al procesar transacciones debido a diversas razones. La manera en que fallan es uno de los factores más importantes para definir si una aplicación es segura o no.
- Se debe contemplar que cuando una aplicación falle, se vuelva igual o más segura de que cuando funciona correctamente.
- *Ejemplo: Para el caso de esta función:*

```
isAdmin = true;  
try {  
    codeWhichMayFail();  
    isAdmin = isUserInRole( "Administrator" );  
}  
catch (Exception ex) {  
    log.write(ex.toString());  
}
```

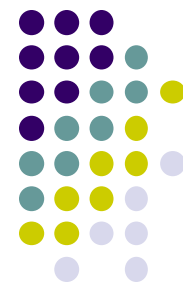
Sí la función codeWhichMayFail() falla, el usuario es administrador por defecto.

Los sistemas externos son inseguros



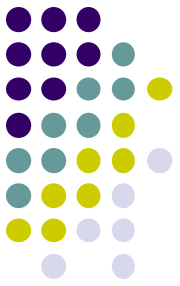
- Diversas organizaciones utilizan las capacidades de procesamiento de terceros, las cuales más que a menudo tienen diferentes políticas de seguridad y posturas que la propia lo que hace muy difícil controlar o influenciar en una tercera parte
- Por esta razón la confianza implícita que podemos tener al ejecutar sistemas internos, no está garantizada para el caso de los externos, por lo que debemos tratar a los mismos como “Sistemas no confiables” y definir los controles en base a este criterio.
- *Por ejemplo, si un proveedor nos proporciona datos que serán utilizados para una aplicación propia, los mismos deberían ser comprobados (en cuanto a longitud, validez, rango etc.) antes de ser utilizados.*

Separación de funciones



- Un control clave para evitar el fraude es la separación de funciones.
 - *Por ejemplo, alguien que se encarga de hacer pedidos de mercancías no debería ser quién realice la recepción de las mismas. Esto previene que esta persona pueda solicitar varios bienes, sustraerlos y luego reclamar que nunca le llegaron.*
- Este es un concepto muy utilizado en la operación segura de los sistemas, para separar las funciones con privilegios especiales (administradores por ej.) de los usuarios normales.
 - *Por ejemplo, un administrador debería ser capaz de apagar y encender el sistema, configurar políticas de contraseñas pero no debería ser capaz de conectarse con un usuario privilegiado, que fuera capaz de realizar transacciones comerciales en nombre de otro.*

No confiar en la seguridad por oscuridad



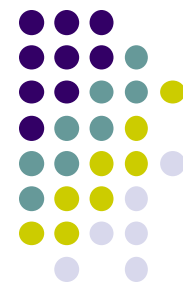
- La seguridad a través de la oscuridad es un control de seguridad débil, que generalmente falla cuando es el único control existente. Esto no significa que no se puedan mantener secretos, sino simplemente que la seguridad de los sistemas clave no debería basarse en detalles ocultos.
- *Por ejemplo, la seguridad de una aplicación no debería basarse en mantener en secreto el conocimiento del código fuente, sino en otros factores, incluyendo políticas razonables de contraseñas, defensa en profundidad, límites en las transacciones de negocios, arquitectura de red sólida, y controles de auditoría y fraude.*

Simplicidad



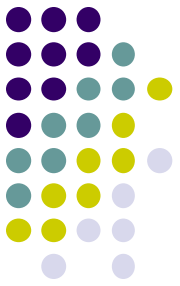
- El área de la superficie de ataque y la simplicidad van de la mano. Ciertos ingenieros de software prefieren aproximaciones demasiado complejas hacia lo que de otra manera sería un código relativamente sencillo y simple.
- Los desarrolladores deben evitar el uso complejas arquitecturas en donde un enfoque simple sería más rápido y simple.

Arreglar cuestiones de seguridad correctamente

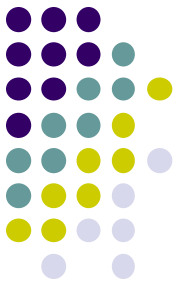


- Una vez que un fallo de seguridad ha sido identificado, es importante efectuar las pruebas necesarias que nos permitan llegar a comprender la raíz del problema.
- Cuando se usan los mismos patrones de diseño, es muy probable que el fallo de seguridad se encuentre muy extendido en todo el código base, por lo que desarrollar la solución correcta sin introducir regresiones es esencial.
- *Por ejemplo, un usuario ha visto que es capaz de ver las cuentas de otro usuario simplemente ajustando su cookie.*
- *La solución parece ser relativamente sencilla, pero como el manejo de la cookie es compartido entre todas las aplicaciones, un cambio en una simple aplicación repercutirá en todas las demás. La solución por lo tanto debe testearse en todas las aplicaciones afectadas.*

¿Que es OWASP?

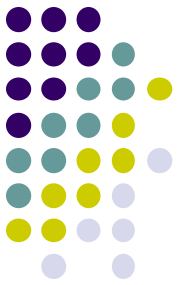


- Open Web Application Application Security Project
 - Promueve el desarrollo de software seguro
 - Orientada a la prestación de servicios orientados a la Web
 - Un foro abierto para el debate
 - Un recurso gratuito para cualquier equipo de desarrollo de software



Qué nos brinda OWASP

- Materiales de Educación
 - OWASP Top 10
 - Guía de Desarrollo OWASP
 - Guía de Testing OWASP
 - Guía OWASP para aplicaciones Web Seguras
- Software
 - WebGoat - JuiceShop
 - ZAP
 - ESAPI
- Capítulos Locales
 - Comunidades interesadas en Seguridad de Aplicaciones



Los 10 riesgos de seguridad más importantes en aplicaciones

OWASP Top Ten

<https://owasp.org/www-project-top-ten/>

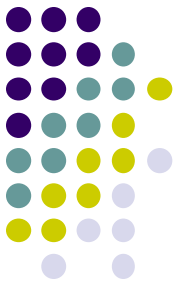


OWASP

The Open Web Application Security Project

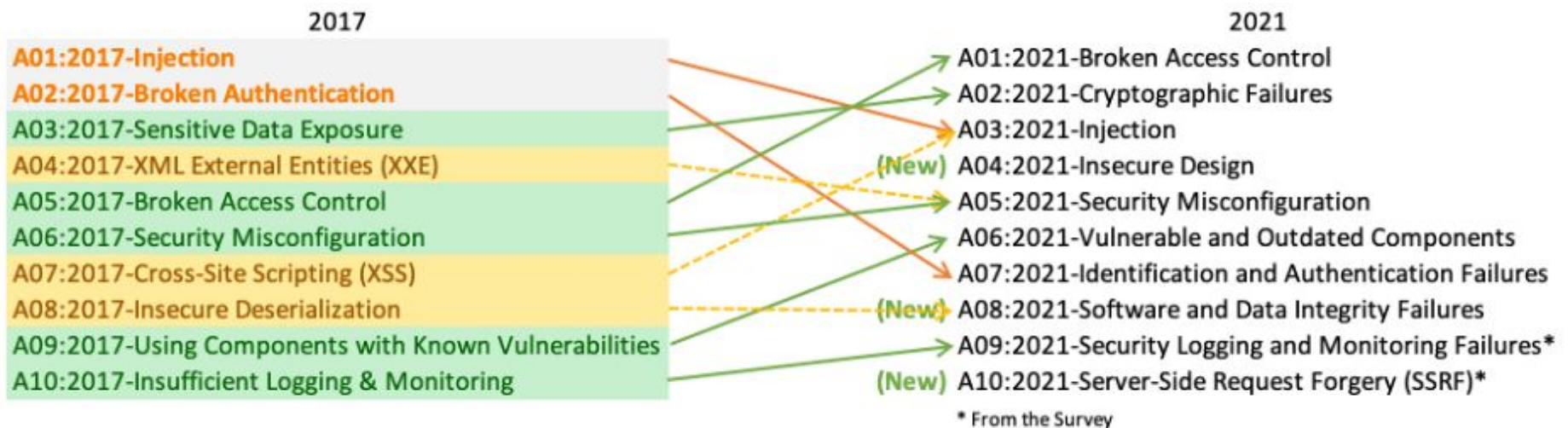
<http://www.owasp.org>

<http://www.owasp.org>



Versiones OWASP top-10

- 2003, 2004, 2007,
- 2010, 2013, 2017,
- 2021



WebGoat (y WebWolf)

The image displays the WebGoat web application interface. At the top, there is a red header with the WebGoat logo and a search bar labeled 'Search lesson'. Below the header, a navigation menu on the left lists various lessons, with 'Developer Tools' highlighted in red. The main content area shows the 'Developer Tools' lesson, which includes a 'Reset lesson' button and a progress indicator with six steps, the fourth of which is active. The lesson title 'Google Chrome Developer Tools' is prominently displayed, followed by a paragraph explaining that certain assignments may require looking at JavaScript source code or running commands, and that Google Chrome provides tools for this purpose. A note at the bottom states that while these tools are not present, users are not obliged to use WebWolf and can use any intercepting tool like netcat.

WEBGOAT

Search lesson

Introduction >

Introduction >

General >

HTTP Basics

HTTP Proxies

Developer Tools

CIA Triad

Writing new lesson

(A1) Broken Access Control >

(A2) Cryptographic Failures >

(A3) Injection >

(A5) Security Misconfiguration >

(A6) Vuln & Outdated Components >

(A7) Identity & Auth Failure >

Reset lesson

1 2 3 4 5 6 ➔

Google Chrome Developer Tools

To complete certain assignments, you sometimes may have to look at the JavaScript source code or run a JavaScript command on your own. To do that, Google Chrome has a set of tools that allow you to do that and much more. While these tools are not present, you are not obliged to use WebWolf. You can also use any intercepting tool you like. (netcat etc.)

ing WebWolf

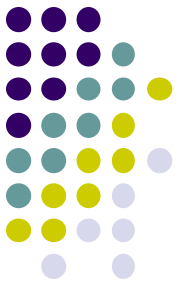
WebWolf if a lesson specifies that you can use it. For use WebGoat without using WebWolf. Lessons WebWolf, are marked with the following icon (top right):

A1 (ex A5) - Broken Access Control

- Hijack Session
- Insecure Direct Object References
- Missing Function Level Access Control
- Spoofing an Authentication Cookie



A1 – Pérdida de Control de Acceso



- La mayoría de las aplicaciones web verifican permisos antes de permitir el acceso a una funcionalidad en la interfaz de usuario.
- Sin embargo, es necesario además que se realicen las mismas comprobaciones de control de acceso en el servidor cuando se accede a cada función ya que si no se verifican todas las peticiones, un atacante puede forzar peticiones con el fin de acceder a la funcionalidad no autorizada.
- Para hacerlo los atacantes podrán alterar parámetros de sesión, o directamente las URLs para acceder a objetos no autorizados.
- Este tipo de vulnerabilidades se encuentran con frecuencia cuando links y botones simplemente se ocultan a usuario no autorizados, pero la aplicación no protege adecuadamente las páginas de destino.



A1 – Pérdida de Control de Acceso

Esta vulnerabilidad puede presentarse cuando un desarrollador expone una referencia a un objeto de implementación interno, tal como un archivo, directorio, o base de datos sin un chequeo de control de acceso. En este caso los atacantes pueden manipular estas referencias para acceder datos no autorizados.

Ejemplo con acceso a una cuenta bancaria:

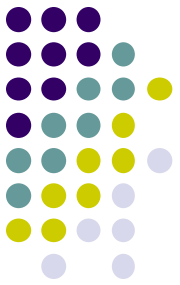
<https://www.homebanking.com/home/app/NroCta=5886800-481-1>

El usuario podría modificar el número de cuenta en la URL

<https://www.homebanking.com/home/app/NroCta=5886801-481-1>

Lo mismo puede pasar con un archivo:

<http://www.finanzas.com/acciones/reporte/pp?file=reporte092011.pdf>



A1 – Pérdida de Control de Acceso

Ejemplo:

Considere las siguientes URLs las cuales se supone que requieren autenticación.

http://ejemplo.com/app/user_getappInfo

Y para acceder a la página “admin_getappInfo” se necesitan permisos de administrador

http://ejemplo.com/app/admin_getappInfo

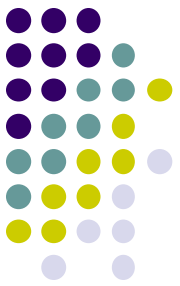
Si la aplicación es vulnerable, un usuario autorizado pero que no sea Administrador puede acceder a la página “admin_getappInfo” a través de la invocación a la URL.

A2 (ex A3 del 2017) - Cryptographic Failures

Crypto Basics - Hijack a session

- Concept:
 - Application developers who develop their own session IDs frequently forget to incorporate the complexity and randomness necessary for security. If the user specific session ID is not complex and random, then the application is highly susceptible to session-based brute force attacks.
- Goal:
 - Gain access to an authenticated session belonging to someone else.





Hijack a session

[Show hints](#)[Reset lesson](#)

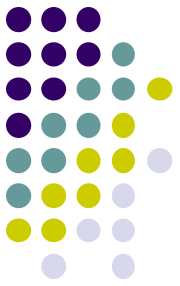
← 1 2

In this lesson we are trying to predict the 'hijack_cookie' value. The 'hijack_cookie' is used to differentiate authenticated and anonymous users of WebGoat.

Account Access



A2 – (A3 en 2017: Exposición de datos sensibles)



- Muchas aplicaciones no protegen adecuadamente los datos sensibles, tales como tarjetas de crédito, datos personales y credenciales de autenticación.
- Los atacantes pueden robar o modificar dichos datos débilmente protegidas para llevar a cabo el robo de identidad, fraudes de tarjetas de crédito, u otros delitos.
- Los datos sensibles necesitan protección adicional, como el cifrado en el almacenamiento y el transporte, así como cuidados especiales cuando se intercambian a través del navegador.

A3 (ex A1 y A7 del 2017) - Injection

- SQL Injection



The screenshot shows the WebGoat interface for the SQL Injection (intro) lesson. On the right, a navigation menu lists: Introduction, General, (A1) Broken Access Control, (A2) Cryptographic Failures, and (A3) Injection. The (A3) Injection menu is expanded, showing sub-items: SQL Injection (intro) (highlighted in red), SQL Injection (advanced), SQL Injection (mitigation), Cross Site Scripting, and Cross Site Scripting (stored). Below the menu, a progress bar shows 13 steps, with step 1 highlighted. The main content area is titled 'Concept' and begins with the text: 'This lesson describes what Structured Query Language (SQL) is and how it can be manipulated to perform tasks that were not the original intent of the developer.'

- Cross Site Scripting (ex A7)

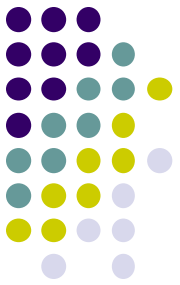
- Path traversal



Path traversal

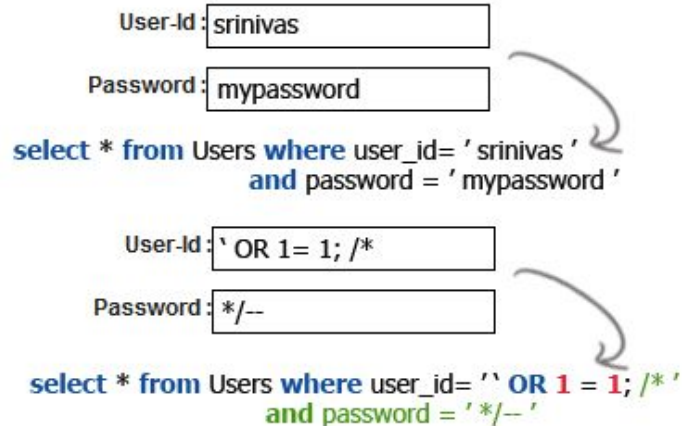
A path(directory) traversal is a vulnerability where an attacker can access or store files and directories outside the application's location. It may lead to reading files from other directories and overwriting critical system files in case of a file upload.

A3 – Injection



- Las fallas de inyección, tales como SQL, OS, y LDAP, ocurren cuando datos no confiables son enviados a un intérprete como parte de un comando o consulta.
- Los datos maliciosos del atacante pueden engañar al intérprete en ejecutar comandos no intencionados o acceder datos no autorizados.

SQL Injection.



Una falla de este tipo puede provocar una pérdida o corrupción de datos, falta de integridad, o denegación de acceso y hasta algunas veces permitir la toma de posesión completa del servidor o la base de datos comprometida

A3 – Cross Site Scripting (XSS)



Un ataque de Cross-Site Scripting consiste en una inclusión de un script en una página Web que se ejecuta cuando la página es accedida por un usuario



Estas fallas ocurren debido a que las aplicaciones toman datos no confiables y los envía al navegador web sin una validación y codificación apropiada. Esto permite a los atacantes ejecutar secuencia de comandos en el navegador de la víctima los cuales pueden secuestrar las sesiones de usuario, destruir sitios web, o dirigir al usuario hacia un sitio malicioso.

A3 – Cross Site Scripting (XSS)



TIPOS DE XSS

1) No permanente o Reflejado

El código de scripting se incluye en el request, generalmente en la URL

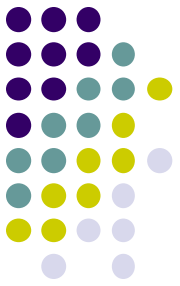
2) Permanente

El código de scripting se almacena en la capa de persistencia de la aplicación (Base de datos o Archivos)

3) Local

Por alguna otra debilidad el atacante logra inyectar el código de scripting en un archivo almacenado localmente en el equipo de la víctima

A4 (not present in 2017) - Insecure Design



Sin presencia en WebGoat (motivo obvio)

A5 (ex A4 y A6 en 2017) - Security Misconfiguration

XXE: XML External Entities

 **WEBGOAT**

XXE

Search lesson

[Introduction](#) >

[General](#) >

[\(A1\) Broken Access Control](#) >

[\(A2\) Cryptographic Failures](#) >

[\(A3\) Injection](#) >

[\(A5\) Security Misconfiguration](#) >

XXE

[\(A6\) Vuln & Outdated Components](#) >

[\(A7\) Identity & Auth Failure](#) >

[\(A8\) Software & Data Integrity](#) >

[\(A9\) Security Logging Failures](#) >

[\(A10\) Server-side Request Forgery](#) >

[Client side](#) >

[Challenges](#) >

Reset lesson

[1](#) [2](#) [3](#) [4](#) [5](#) [6](#) [7](#) [8](#) [9](#) [10](#) [11](#) [12](#) [13](#) ➔

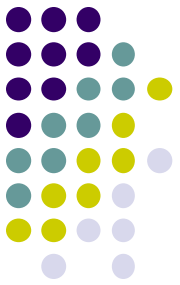
Concept

This lesson teaches how to perform an XML External Entity attack and how it can be abused and protected against.

Goals

- The user should have basic knowledge of XML
- The user will understand how XML parsers work
- The user will learn to perform an XXE attack and how to protected against it.

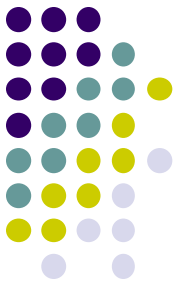
A5 – Entidades Externas XML (XXE)



Entidades Externas XML (XXE)

- Las restricciones sobre lo que los usuarios autenticados pueden hacer no se aplican correctamente. Los atacantes pueden explotar estos defectos para acceder, de forma no autorizada, a funcionalidades y/o datos, cuentas de otros usuarios, ver archivos sensibles, modificar datos, cambiar derechos de acceso y permisos, etc.
- Los atacantes pueden explotar procesadores XML vulnerables, incluyendo contenido hostil en un documento XML o explotando código vulnerable, dependencias o integraciones.
- Estas deficiencias se pueden utilizar para extraer datos, ejecutar una solicitud remota desde el servidor, escanear sistemas internos, o realizar un ataque de denegación de servicio.

A5 – Entidades Externas XML (XXE)



Ejemplos

Escenario #1: el atacante intenta extraer datos del servidor:

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE foo [
<!ELEMENT foo ANY>
<!ENTITY xxe SYSTEM "file:///etc/passwd">]>
<foo>&xxe;</foo>
```

Escenario #2: cambiando la línea *ENTITY* anterior, un atacante puede escanear la red privada del servidor:

```
<!ENTITY xxe SYSTEM "https://192.168.1.1/private">]>
```

Escenario #3: incluyendo un archivo potencialmente infinito, se intenta un ataque de denegación de servicio:

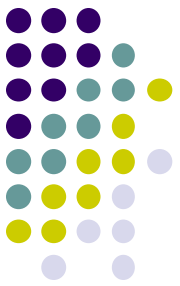
```
<!ENTITY xxe SYSTEM "file:///dev/random">]>
```

A5 – Configuración de seguridad incorrecta



- Una buena estrategia de seguridad requiere tener definida e implementada una configuración segura para la aplicación, framework , servidor de aplicación, servidor web, base de datos y plataforma.
- Todas estas configuraciones deben ser definidas, implementadas, y mantenidas ya que por lo general no son seguras por defecto.
- Esto incluye mantener todo el software actualizado, incluidas las librerías de código utilizadas por la aplicación.
- Ejemplos de configuración insegura:
 - Cuentas por defecto
 - Páginas de ejemplo en desuso
 - Consola de administración activas y accesibles
 - Falta de actualización de patches
 - Seteos de configuración por defecto
 - Claves de cifrado débiles.

A6 (ex A9 en 2017) - Vulnerable and Outdated Components



En 2017: “Usin Components with Known Vulnerabilities”

The screenshot displays the WebGoat application interface. On the left is a navigation sidebar with a red header containing a goat logo and the text 'WEBGOAT'. The sidebar lists various lessons, with 'Vulnerable Components' highlighted in red. The main content area has a white header with the word 'Vulnerable' and a search bar. Below the header are buttons for 'Show hints' and 'Reset lesson'. A progress bar shows 13 lessons, with lesson 5 highlighted in green and lesson 12 in red. The main content area displays the title 'Concept' and a paragraph of text about software development and open source libraries, followed by a reference to 'Software Supply Chain'.

WEBGOAT

Introduction >

General >

(A1) Broken Access Control >

(A2) Cryptographic Failures >

(A3) Injection >

(A5) Security Misconfiguration >

(A6) Vuln & Outdated Components >

Vulnerable Components

(A7) Identity & Auth Failure >

(A8) Software & Data Integrity >

(A9) Security Logging Failures >

(A10) Server-side Request Forgery >

Client side >

Vulnerable

Search lesson

Show hints Reset lesson

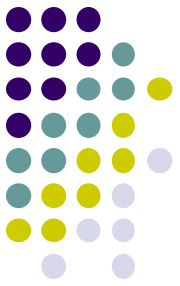
1 2 3 4 5 6 7 8 9 10 11 12 13 ➔

Concept

The way we build software has changed. The open source community is maturing and the availability of open source software has become prolific without regard to determining the provenance of the libraries used in our applications. Ref: [Software Supply Chain](#)

This lesson will walk through the difficulties with managing dependent libraries, the risk of not managing those dependencies, and the difficulty in determining if you are at risk.

A6 – Uso de componentes vulnerables

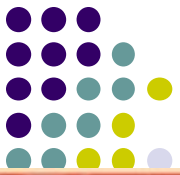


Los componentes de software como bibliotecas, frameworks y otros módulos casi siempre se ejecutan con privilegios elevados. Por lo tanto si alguno presenta una vulnerabilidad y esta es explotada, se logra comprometer el servidor.

Si bien un proceso de gestión de vulnerabilidades mitiga en parte este problema, en la práctica no es sencillo conocer en todos los casos si usted está usando un componente de software vulnerable, ya que no todas las vulnerabilidades son reportadas en el momento que se presenta y muchas veces pasan meses hasta que se hacen conocidas.

Es por ello que se recomienda implementar medidas mitigantes complementarias como puede ser la asignación de mínimos privilegios, el monitoreo de actividad anormal, y la implementación de medidas de seguridad adicionales en la infraestructura para minimizar el riesgo de ser víctima de estos ataques.

A7 (ex A2 del 2017) - Identification and Authorization Failures

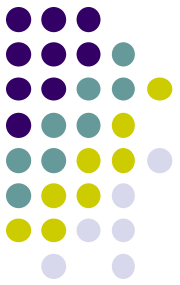


- Authentication Bypasses
- Insecure Login
- JWT tokens
- Password reset
- Secure passwords



Introduction	>
General	>
(A1) Broken Access Control	>
(A2) Cryptographic Failures	>
(A3) Injection	>
(A5) Security Misconfiguration	>
(A6) Vuln & Outdated Components	>
(A7) Identity & Auth Failure	>
Authentication Bypasses	
Insecure Login	
JWT tokens	
Password reset	
Secure Passwords	
(A8) Software & Data Integrity	>

A7 – Pérdida de Autenticación y Manejo de Sesión

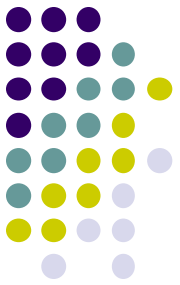


Por la naturaleza del protocolo HTTP que no es orientado a conexión, se hace necesario implementar mecanismos de manejo de sesión entre el cliente y el servidor que si no son implementados correctamente pueden ser alterados para obtener acceso no autorizado a los recursos.



Las funciones de la aplicación relacionadas a autenticación y gestión de sesiones son frecuentemente implementadas incorrectamente, permitiendo a los atacantes comprometer contraseñas, claves, o id de sesiones, o explotar otras fallas de implementación para asumir la identidad de otros usuarios.

A7 – Pérdida de Autenticación y Manejo de Sesión



Métodos de ataque:

Predicción de sesiones

- Generalmente cuando se puede predecir un ID de sesión
- Permite crear un ID de sesión válido de otro usuario

Reutilización de sesiones

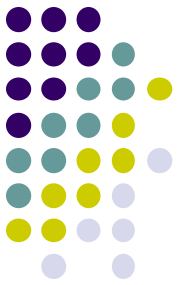
- Cuando un ID de sesión caducó se reutiliza en poco tiempo

Fijación de sesiones

- Cuando se puede generar un ID de sesión arbitrario desde el cliente

Sniffing de identificadores de sesión

- Si el canal no está encriptado
- Si el ID de sesión está en la URL



JWT: JSON Web Tokens

Errores comunes:

- firma inválida, token (tampañado, expirado, malformado, mal validado...), algoritmo (elegido/ible), ataque repeat,...

(A2) Cryptographic Failures >

(A3) Injection >

(A5) Security Misconfiguration >

(A6) Vuln & Outdated Components >

(A7) Identity & Auth Failure >

Authentication Bypasses

Insecure Login

JWT tokens

Password reset

Secure Passwords

(A8) Software & Data Integrity >

(A9) Security Logging Failures >

(A10) Server-side Request Forgery >

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 →

Concept

This lesson teaches about using JSON Web Tokens (JWT) for authentication and the common pitfalls you need to be aware of when using JWT.

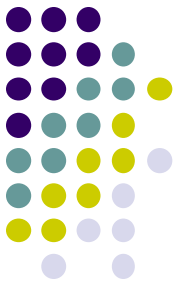
Goals

Teach how to securely implement the usage of tokens and validation of those tokens.

Introduction

Many application use JSON Web Tokens (JWT) to allow the client to indicate its identity for further exchange after authentication.

A8 (~A8 en 2017) - Software and Data Integrity Failures



 **WEBGOAT**

Introduction >

General >

(A1) Broken Access Control >

(A2) Cryptographic Failures >

(A3) Injection >

(A5) Security Misconfiguration >

(A6) Vuln & Outdated Components >

(A7) Identity & Auth Failure >

(A8) Software & Data Integrity >

Insecure Deserialization

(A9) Security Logging Failures >

(A10) Server-side Request Forgery >

Client side >

Challenges >

Insecure D



Show hintsReset lesson

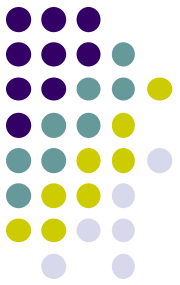
12345➔

Concept

This lesson describes what is Serialization and how it can be manipulated to perform tasks that were not the original intent of the developer.

Goals

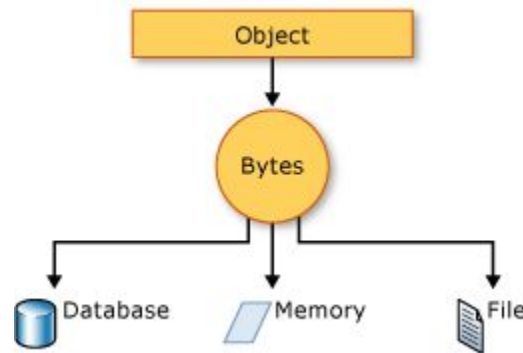
- The user should have a basic understanding of Java programming language
- The user will be able to detect insecure deserialization vulnerabilities
- The user will be able to exploit insecure deserialization vulnerabilities
- Exploiting deserialization is slightly different in other programming languages such as PHP or Python, but the key concepts learnt here also applies to all of them

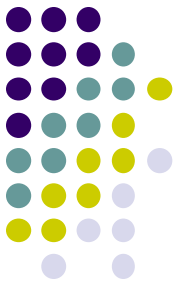


A8 – Deserealización insegura

- **¿Qué es la serialización?**

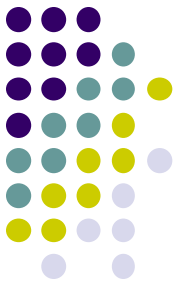
La serialización es el proceso de codificación de un objeto (como un archivo o buffer de memoria) en una secuencia de bytes para almacenarlo o transmitirlo a un dispositivo o base de datos en un formato legible como JSON o XML (entre otros).





A8 – Deserealización insegura

- La falla de seguridad de deserialización se produce cuando los datos serializados que el atacante envía al servidor de consumo de servicios, hacen que se ejecute al momento de deserializar, un código arbitrario (malicioso).
- El riesgo de esta falla de seguridad es que el atacante, gracias al código malicioso, puede tomar control del servidor remotamente u obtener información de este. En específico serían los siguientes ataques:
- Ataques relacionados con la estructura de datos y objetos donde el atacante modifica la lógica de la aplicación o logra una ejecución remota de código si existen clases disponibles para la aplicación que pueden cambiar el comportamiento durante o después de la deserialización.
- Ataques típicos de manipulación de datos, como ataques relacionados con el control de acceso en los que se utilizan estructuras de datos existentes pero se modifica su contenido.



A8 – Deserealización insegura

- **Ejemplo**

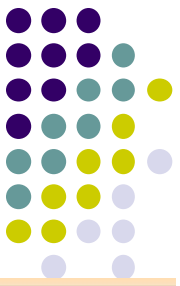
Una aplicación PHP utiliza serialización de objetos PHP para almacenar una cookie, conteniendo el ID, rol, hash de la contraseña y otros estados del usuario:

- `a:4:{i:0;i:132;i:1;s:7:"Mallory";i:2;s:4:"user";i:3;s:32:"b6a8b3bea87fe0e05022f8f3c88bc960";}`

Un atacante modifica el objeto serializado para darse a si mismo los privilegios de administrador:

- `a:4:{i:0;i:1;i:1;s:5:"Alice";i:2;s:5:"admin";i:3;s:32:"b6a8b3bea87fe0e05022f8f3c88bc960";}`

A9 (A10 en 2017) - Security Logging and Monitoring Failures



**WEBGOAT**

Logging Se



Introduction >

General >

(A1) Broken Access Control >

(A2) Cryptographic Failures >

(A3) Injection >

(A5) Security Misconfiguration >

(A6) Vuln & Outdated Components >

(A7) Identity & Auth Failure >

(A8) Software & Data Integrity >

(A9) Security Logging Failures >

Logging Security

(A10) Server-side Request Forgery >

Client side >

Challenges >

Reset lesson

1

2

3

4

5

➔

Concept

Logging is very important for modern systems. We use it for various reasons:

- Application monitoring and debugging.
- Audit logging: E.g. record specific actions of your users and systems.
- Security Event Monitoring: e.g. provide information to a SIEM or SOAR system that will trigger based on the information provided in these logs.

Goals

- The user should have a basic understanding of logging and where to log for.
- The user understands the risks of log spoofing and leaking log information.
- The user will be able to do a simple log spoofing attack.



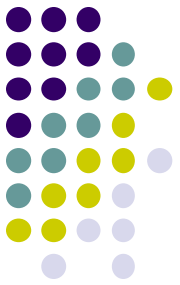
A10 - Registro y Monitoreo Insuficientes

El registro y monitoreo insuficientes es la base de casi todos los grandes y mayores incidentes de seguridad ya que los atacantes dependen de la falta de monitoreo y respuesta oportuna para lograr sus objetivos sin ser detectados.

Una estrategia para determinar se cuenta con el monitoreo suficiente es examinar los registros después de las pruebas de penetración. Las acciones de los evaluadores deben registrarse lo suficiente como para comprender los daños que podrían haber causado.

La falta de estos aspectos puede impedir o demorar en forma significativa la detección de actividad maliciosa o de sustracción de datos, la respuesta a los incidentes y la investigación forense digital.

A10 (not in 2017) - SSRF: Server Side Request Forgery

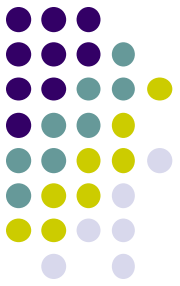


- Cross Site Request Forgeries
- Server Side Request Forgery

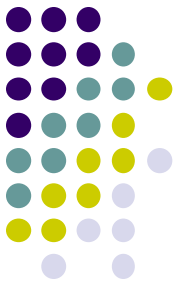


Introduction	>
General	>
(A1) Broken Access Control	>
(A2) Cryptographic Failures	>
(A3) Injection	>
(A5) Security Misconfiguration	>
(A6) Vuln & Outdated Components	>
(A7) Identity & Auth Failure	>
(A8) Software & Data Integrity	>
(A9) Security Logging Failures	>
(A10) Server-side Request Forgery	>
Cross-Site Request Forgeries	
Server-Side Request Forgery	
Client side	>
Challenges	>

Fallas de Seguridad en aplicaciones OWASP



- Ver Documento :
 - <https://www.owasp.org/images/5/5e/OWASP-Top-10-2017-es.pdf>



Buenas prácticas de Programación OWASP

- Ver Documentos :
- OWASP Top 10 Proactive Controls 2016
 - https://www.owasp.org/images/5/57/OWASP_Proactive_Controls_2.pdf
- Controles Proactivos OWASP 2016
 - <https://es.slideshare.net/henryraulgonzalez/10-principales-controles-pro-activos-de-owasp>